

```
In [5]: from googlefinance.client import get_price_data, get_prices_data
        from scipy.optimize import minimize
        from matplotlib.mlab import normpdf

        import matplotlib.pyplot as plt
```

Portfolio optimization

We consider the problem of choosing a *long term* portfolio, given a set of stocks and their price over some period under *risk aversion parameter*, $\gamma > 0$.

The *portfolio* will be represented by a vector w , such that $\sum_{i=1}^n w_i = 1$. $w_i > 0$ represent using w_i fraction of your total money to *buy* the i' th stock, while $w_i < 0$ represent *shorting* that stock. In both cases the stock is bought/shorted for the entire period.

$p_{j,i}$ represent the price of the i' th stock at timestep j . That is, $p \in \mathbb{R}^{n \times m}$ is a matrix where n is the number of timesteps and m is the number of stocks to consider.

$r_{j,i}$ represent the fractional reward of stock i at timestep j . so, $r_{j,i} = \frac{p_{j,i} - p_{j-1,i}}{p_{j,i}}$ for $j > 1$.

Now we model r as a *Random Variable*, distributed as a multivariate gaussian with means,

$$\mu = \mathbb{E}[r] = \frac{1}{n} \sum_{j=1}^n r_{j,i}$$

and variance,

$$\Sigma = \mathbb{E}[(r - \mu)(r - \mu)^T] = \frac{1}{n} \sum_{j=1}^n [(r_j - \mu)(r_j - \mu)^T].$$

The distribution of *Returns* using some w is then just,

$$R_w = \mathcal{N}(\mu_w, \sigma_w), \quad \mu_w = w^T \mu, \quad \sigma_w = w^T \Sigma w.$$

Now, naturally we want to *maximize* the expected returns μ_w while *minimizing* the risk, σ_w . This leads to the following optimization problem:

$$w^* = \arg \max_w \left[w^T \mu - \gamma w^T \Sigma w \right] \\ \text{st. } \sum w_i = 1.$$

Where γ control the amount of risk we are willing to take. A high value of γ indicate we are willing to take *low* risks and vice versa.

Your job is to find this w^* using real stock values of your choosing, under different values of γ . To make your job easier, the task is seperated into the following subtasks:

1. We need a module for collecting stockvalues, for this we will use `googlefinance`. See [here](https://pypi.org/project/googlefinance/) (<https://pypi.org/project/googlefinance/>) for a description of how to install and use the module. Using this you should write a function `get_prices([stocks, ...], step_size, period, exchange)`, that return a tuple, `(stocks, p)`. Where `p[i, j]` represent the **opening** price of stock j at timestep i and `stocks[j]` is the name of the j' th stock. You should document this function by including a plot of p , where each stock is labeled with its name (for example `MSFT` or `GGL`). you should use atleast 5 stocks to get good results.
2. Calculate r , μ and Σ using the formulars above and the p you calulated at (1). Plot the *pdf* of each stock. (**Hint:** `from matplotlib.mlab import normpdf` might be handy).
3. Solve the optimization problem defined above for different values of gamma (for example `gammas = (np.arange(10) / 5) + 1`) and plot the *pdf* of each solution to a single plot with aproprate legends, Also create a scatterplot of how w^* change as γ change.

In [6]: # SOLUTION TO 1

```
def get_prices(stocks=['MSFT', 'GOGL', 'FLWS', 'FOX', 'UHAL'], step_size=6400, period=250):
    params = [{'q': stock, 'i': step_size, 'x': exchange} for stock in stocks]
    df = get_prices_data(params, period)

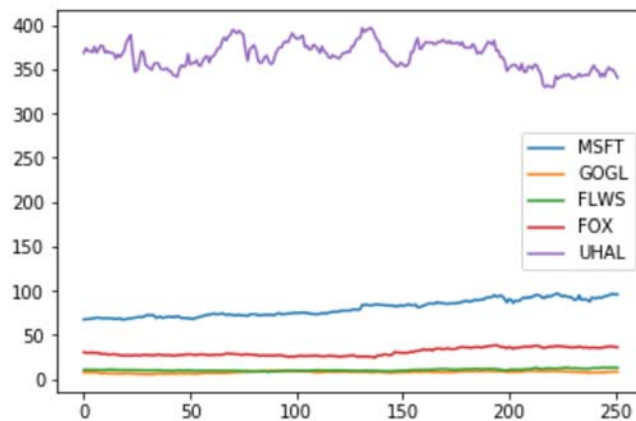
    keys = [key for key in df.keys() if "_Open" in key]

    return [key.replace('_Open', '') for key in keys], df.as_matrix(keys)

stocks, p = get_prices()

for name, data in zip(stocks, p.T):
    plt.plot(data, label=name)

plt.legend()
```



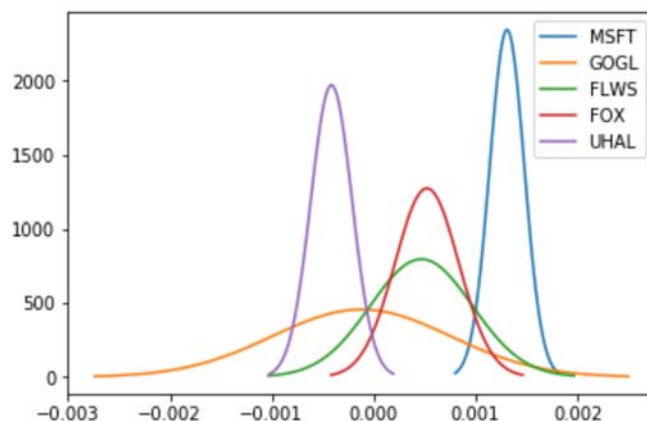
In [7]: # SOLUTION TO 2

```
r = ((p - np.roll(p, 1, axis=0)) / p)[1:]
means = r.mean(axis=0)
sigma = sum(np.outer(r_ - means, r_ - means) for r_ in r) / r.shape[0]

for i in range(means.size):
    m, s = means[i], sigma[i, i]
    x = np.linspace(m - 3 * s, m + 3 * s, 100)

    plt.plot(x, normpdf(x, m, s), label=stocks[i])

plt.legend()
```



```

In [8]: # SOLUTION TO 3

gammas = (np.arange(1, 10) / 5) + 1

def get_dist(x):
    m = x.T @ means
    s = x.T @ sigma @ x
    x_ = np.linspace(m - 3 * s, m + 3 * s, 100)
    return x_, normpdf(x_, m, s)

def objective_fun(gamma):
    def objective(w):
        return -(w.T @ means - gamma * w.T @ sigma @ w)
    return objective

solutions = np.array(
    list(
        i['x'] for i in (
            minimize(
                objective_fun(gamma_),
                np.array([1/len(stocks) for _ in stocks]),
                constraints={'type': 'eq', 'fun': lambda w: sum(w) - 1}
            )
            for gamma_ in gammas
        )
    )
)

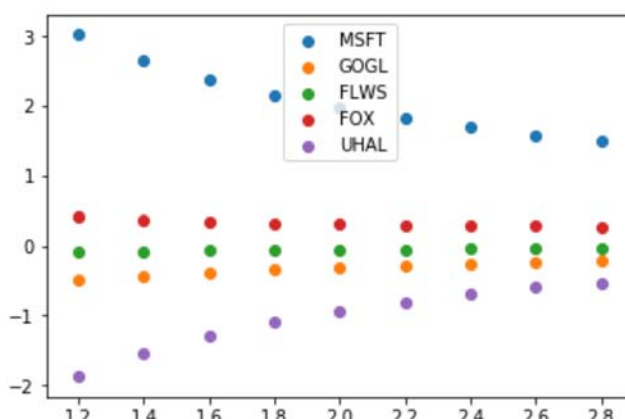
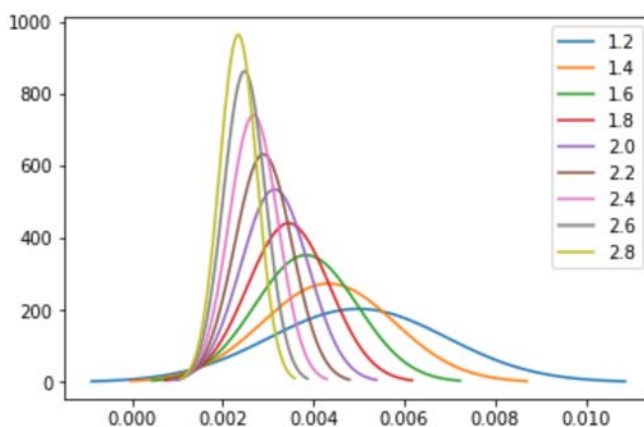
for gamma, data in zip(gammas, solutions):
    plt.plot(*get_dist(data), label=str(gamma))

plt.legend()
plt.show()

for i in range(solutions.shape[1]):
    plt.scatter(gammas, solutions[:,i], label=stocks[i])

plt.legend()

```



In []: